

Restaurarea și Clasificarea Inscripțiilor Oracol

Proiect Practic | Enunț al Problemei & Specificație Tehnică

Versiunea 2.0 — Bază de Date Reală & Clasificare pe Tiers

Cuprins

1	Configurația Problemei & Date de Intrare	1
2	Task 1: Filtrarea Frecvențelor Spațiale (Fourier 2D) — 30p	1
3	Task 2: Decuplarea Caracteristicilor (Wavelet Haar 2D) — 20p	2
4	Task 3: Extragerea Asimetrică & Gradientul Direcțional — 20p	2
5	Task 4: SVD, Hashing Binar & Clasificare Hamming — 20p	2
6	Structura Testelor & Setul de Date (Practice vs Competition)	3
7	Evaluare, Bareme & Punctaj	3
8	Constrângeri Tehnice & Reguli Stricte (STRICT INTERZIS)	3
9	Instrucțiuni de Trimitere & Makefile	3

1 Configurația Problemei & Date de Intrare

Fiecare test furnizează o matrice pătratică $A^{N \times N}$ (cu $N = 64$), reprezentând intensitatea pixelilor normalizată în intervalul $[0, 1]$ dintr-o imagine degradată a unui caracter antic oracol (dinastia Shang, datasetul Oracle-MNIST). Suplimentar, este disponibil un dicționar de referință `data/known_symbols/` conținând $C = 10$ simboluri prototip reale.

Obiectivul: Implementarea unui flux numeric complet de procesare a semnalului pentru extragerea trăsăturilor geometrice invariante și clasificarea imaginii în una dintre cele $C = 10$ clase prin distanță Hamming minimă.

Parametri globali: Dimensiunea imaginii $N = 64$, dimensiunea spațiului de trăsături / hash $K = N / 2 = 32$, numărul de clase de caractere $C = 10$.

2 Task 1: Filtrarea Frecvențelor Spațiale (Fourier 2D) [30 puncte]

Pentru a elimina zgomotul de înaltă frecvență cauzat de porozitatea osului și degradarea fizică, imaginea este mapată în domeniul frecvenței folosind Transformata Fourier Discretă 2D (DFT 2D).

- Subtask 1.1 (10p) — Construcția matricei Fourier 2D:

Construiți matricea exponențialelor complexe $F^{N \times N}$ conform formulei:

$$F_{m,n} = \exp(-2\pi i \cdot m \cdot n / N), \text{ pentru } m, n \in \{0, 1, \dots, N-1\}.$$

Calculați spectrul bidimensional al imaginii prin transformarea matriceală: $X = F \cdot A \cdot F$.

- Subtask 1.2 (10p) — Masca binară trece-jos (Low-Pass Filter):

Generați o mască binară $M^{N \times N}$ astfel încât elementele din submatricea centrală de dimensiune $(N/2) \times (N/2)$ (adică liniile și coloanele din intervalul $N/4 + 1 : 3N/4$) să fie 1, iar restul 0.

Aplicați filtrarea spectrală prin produsul Hadamard: $X_f = X \cdot M$.

- Subtask 1.3 (10p) — Reconstrucția imaginii filtrate:

Reconstruiți imaginea spațială filtrată $\tilde{A}^{N \times N}$ aplicând transformata inversă 2D:

$$\tilde{A} = \text{Re}(F^{-1} \cdot X_f \cdot F^{-1}), \text{ unde } F^{-1} = (1/N) \cdot F^* \text{ (conjugata hermitică)}.$$

3 Task 2: Decuplarea Caracteristicilor (Wavelet Haar 2D) [20 puncte]

Pentru a decupla componentele direcționale ale trăsăturilor (linii orizontale vs verticale), se aplică Transformata Wavelet Discretă 2D (DWT 2D) utilizând baza ortonormată Haar.

- Subtask 2.1 (10p) — Construcția operatorului Haar:

Construiți matricea ortogonală Haar $H^{N \times N}$ definită pe blocuri:

Pentru $i = 1, \dots, K$ (cu $K = N/2$):

$$H(i, 2i-1) = 1/\sqrt{2}, \quad H(i, 2i) = 1/\sqrt{2}$$

$$H(K+i, 2i-1) = 1/\sqrt{2}, \quad H(K+i, 2i) = -1/\sqrt{2}$$

Calculați coeficienții Wavelet 2D prin transformarea: $W = H \cdot \tilde{A} \cdot H^T$.

- Subtask 2.2 (10p) — Extragerea subbenzilor direcționale:

Extrageți din matricea W cele două submatrice de dimensiune $K \times K$:

1. W_{LH} (cadranul superior-dreapta, linii 1:K, coloane K+1:N) — variații orizontale.
2. W_{HL} (cadranul inferior-stânga, linii K+1:N, coloane 1:K) — variații verticale.

4 Task 3: Extragerea Asimetrică & Gradientul Direcțional [20 puncte]

Deoarece trăsăturile caligrafice oracol prezintă anizotropie puternică, aplicăm diferențierea numerică asimetrică folosind scriptul de laborator Dif.m.

- Subtask 3.1 (10p) — Derivare numerică direcțională:

Calculați derivata numerică de ordinul 1 pe fiecare linie a matricei W_{LH} pentru a obține componenta orizontală $G_x^{K \times K}$, și pe fiecare coloană a matricei W_{HL} pentru a obține componenta verticală $G_y^{K \times K}$.

Se va utiliza schema cu diferențe finite centrate la interior și diferențe unilaterale la capete (conform Dif.m).

- Subtask 3.2 (10p) — Magnitudinea gradientului trăsăturilor:

Sintetizați harta bidimensională a trăsăturilor calculând magnitudinea euclidiană element cu element:

$$S = \sqrt{(G_x^2 + G_y^2)}^{K \times K}.$$

5 Task 4: SVD, Hashing Binar & Clasificare Hamming [20 puncte]

Comprimați matricea S într-o amprentă binară compactă și clasificați caracterul prin distanță Hamming minimă față de dicționar.

- Subtask 4.1 (10p) — Descompunerea în Valori Singulare (SVD):

Efectuați descompunerea SVD a matricei trăsăturilor: $S = U \cdot \Sigma \cdot V^T$ folosind algoritmul numeric dezvoltat la laborator (SVD.m). Extrageți diagonala principală a matricei Σ pentru a forma vectorul valorilor singulare v^K .

- Subtask 4.2 (5p) — Binarizare și Hashing:

Calculați media valorilor singulare $\mu = \text{mean}(v)$. Generați codul binar de hash $b \in \{-1, +1\}^K$ aplicând funcția semn:

$$b_i = \text{sign}(v_i - \mu), \quad (\text{unde } \text{sign}(0) \text{ este mapat la } +1).$$

- Subtask 4.3 (5p) — Clasificare prin Distanță Hamming Minimă:

Având la dispoziție matricea known_hashes $\{-1, +1\}^{K \times C}$ (conținând codurile de hash ale celor $C = 10$ simboluri cunoscute), calculați distanța Hamming față de fiecare coloană $c \in \{1, \dots, C\}$:

$$d_c = \sum_{i=1..K} [b_i \neq \text{known_hashes}(i, c)].$$

Identificați clasa prezisă: $\text{predicted_class} = \arg\min_c (d_c)$ și distanța minimă asociată $\text{min_dist} = \min(d)$.

6 Structura Testelor & Setul de Date

Evaluarea temei folosește două seturi de date distincte pentru a măsura robustețea numerică și a preveni supra-optimizarea (hardcodarea):

Set / Tier	Nr. Imagini	Tip Zgomot / Degradare	Scop Evaluare
Practice — Tier 1	10	Contrast optim, zgomot minim	Validare de bază algoritmi
Practice — Tier 2	10	Textură de piatră & porozitate	Sensibilitate la variații de tuș
Practice — Tier 3	10	Zgomot speckle & micro-eroziune	Eficiență Fourier & Wavelet
Competition (Secret)	40	4 Tiers (inclusiv Extra-Hard)	Evaluare oficială anti-hardcodare

7 Evaluare și Punctaj

Punctajul maxim total este de 100 puncte, distribuit astfel:

Componentă	Punctaj	Criterii de Validare
Task 1 (Fourier 2D)	30 puncte	Matrice F (10p), Mască M & X _f (10p), Reconstrucție A _{tilde} (10p)
Task 2 (Wavelet Haar)	20 puncte	Matrice Haar H & W (10p), Subbenzi W _{LH} & W _{HL} (10p)
Task 3 (Derivare Numerică)	20 puncte	Gradienți G _x & G _y (10p), Magnitudine S (10p)
Task 4 (SVD & Clasificare)	20 puncte	Valori singulare v (10p), Hash b (5p), Clasificare Hamming (5p)
Coding Style & README	10 puncte	Comentarii clare, cod vectorizat, structură modulară
TOTAL	100 puncte	90p Parțial (Checker) + 10p Stil / Barem Manual

8 Constrângeri Tehnice & Reguli Stricte (STRICT INTERZIS)

Scopul temei este aprofundarea algoritmilor numerici la nivel de bază. Utilizarea funcțiilor built-in automate de nivel înalt anulează valoarea pedagogică și este STRICT INTERZISĂ în soluțiile submise.

Sunt STRICT INTERZISE în fișierele din src/:

- Descompuneri spectrale și factorizări built-in: svd, svds, eig, eigs, schur.
- Transformate Fourier și Wavelet automate: fft, fft2, fftn, ifft, ifft2, dwt, dwt2, idwt2, wavedec2.
- Derivare și convoluție automată: diff, gradient, conv, conv2, filter2, imfilter, edge.
- Toolbox-uri specializate: Image Processing Toolbox, Signal Processing Toolbox, Wavelet Toolbox, Statistics / Machine Learning Toolbox.
- Solvatori de sisteme pe matrici mari: inv, pinv (este permisă doar inversarea explicită cu formula $F^{-1} = (1/N) \cdot F^*$).

Sunt PERMISE și RECOMANDATE:

- Operații aritmetice matriceale primitive: multiplicare (*), produs pe elemente (.*), transpunere (', .'), conjugare (conj), norme (norm), sume (sum), rădăcină (sqrt), funcții trigonometrice/exponențiale (exp, sin, cos).
- Reutilizarea algoritmilor dezvoltati la laboratoare: Dif.m (Laborator 10) și SVD.m (Laborator 7).

9 Instrucțiuni de Trimitere & Makefile

Pentru a asigura o testare și o livrare fără erori, folosiți comenzile definite în Makefile:

- make check — Rulează checker-ul automat în Octave pe setul de practică (data/practice).
- make docker-check — Construiește containerul Docker izolat și execută suita completă de teste.
- make submit — Împachetează exclusiv fișierele sursă cerute din src/ în arhiva submission.zip.
- make clean — Șterge fișierele și arhivele temporare generate.

Atenție: Arhiva submission.zip va fi evaluată automat pe un mediu curat Linux Octave peste datasetul secret data/competition/. Asigurați-vă că soluția voastră rulează fără erori și respectă semnăturile funcțiilor cerute!